Energy-Aware Scenario-Based Mapping of Deep Learning Applications Onto Heterogeneous Processors Under Real-Time Constraints

Jangryul Kim and Soonhoi Ha, *Fellow, IEEE*

Abstract—To cope with the increasing demand for deep learning applications in embedded systems, emerging embedded devices tend to equip multiple heterogeneous processors, including GPU and deep learning hardware accelerator, called neural processing unit (NPU). It becomes popular to run multiple deep learning (DL) applications simultaneously to provide several functionalities. In this work, we assume that applications have real-time constraints that may vary at run time. While extensive studies have been conducted recently to find an efficient mapping of multiple DL applications on various hardware platforms, they do not consider the constraints imposed by the NPU and the associated software development kit (SDK) in a real embedded platform. In this paper, we propose a novel energy-aware mapping methodology of multiple DL applications onto a real embedded system that has multiple heterogeneous processors. The objective is to minimize energy consumption while satisfying the real-time constraints of all applications. In the proposed scheme, we first select Pareto-optimal mapping solutions for each application. Then mapping combination is explored, considering the scenario that indicates the dynamism of applications while satisfying the constraints. Also, we reduce energy consumption by tuning the frequency of processors. We could satisfy up to 40% higher deadline constraints and reduce the energy consumption by 22% $\sim$ 31% compared to the static mapping methods with real-life applications and different scenarios on a real platform.

Index Terms—Deep learning applications, scenario-based design methodology, application mapping, schedulability test

1 Introduction

Deep learning (DL) applications are becoming more popular in embedded systems. The increasing demand for DL applications is driving embedded devices to equip deep learning hardware accelerators, called neural processing units (NPUs), besides a multi-core CPU and a GPU. A well-known example is NVIDIA Jetson AGX Xavier that contains a CPU, GPU, and two deep learning accelerators (DLAs), which is the assumed hardware platform in this paper. A DLA is a kind of NPU. On the other hand, it becomes more popular to run multiple DL applications parallel in an embedded system. While extensive studies have been conducted recently to find an efficient mapping of multiple DL applications on various hardware platforms ([1], [2], [3], [4]), they do not consider NPUs or have assumptions that are not met in real implementation. In this paper, we propose a novel energy-aware mapping methodology of multiple DL applications onto a real embedded system that has multiple heterogeneous processors. The objective is to minimize energy consumption while satisfying the real-time constraints of all applications.

There exist several technical challenges involved in this mapping problem. First, DL applications usually have real-time constraints in terms of latency, and the real-time constraints may vary at run time. For instance, suppose that a self-driving car runs four object detection networks for the streaming images received from the four sides: front, back, left, and right. As the car speed increases, the latency constraint will be tightened. Second, the set of concurrently running applications may vary. If the car is moving forward, we may want to suspend the object detection from the backside. To handle such dynamism of multiple applications, we define *scenarios* that tell which applications are running.

The third challenge is to estimate the performance of each mapping candidate. In most previous works, it is assumed that the worst-case execution time (WCET) of each layer in a DL application is known, and the performance of mapping can be estimated analytically. But layer-wise profiling is often not appropriate in real systems. An embedded device is accompanied by a software development kit (SDK) that helps a user develop a DL application. TensorRT [5] is the SDK for an NVIDIA device. TensorRT internally optimizes the inference by applying the techniques such as layer fusion which combines multiple consecutive layers mapped on the same processor to a single kernel. Furthermore, additional kernels can be inserted when the network is partitioned to GPU and DLAs. This makes the problem difficult because the profiled per-layer execution time may not be available in

• *The authors are with the Department of Computer Science and Engineering, Seoul National University, Seoul 08826, Korea. E-mail: {urmydata, sha}@snu.ac.kr.*

Manuscript received 17 March 2022; revised 13 October 2022; accepted 30 October 2022. Date of publication 2 November 2022; date of current version 10 May 2023.
This work was supported by Samsung Electronics Co., Ltd. under Grant IO201210-07941-01.
(Corresponding author: Soonhoi Ha.)
Recommended for acceptance by C. Silvano.
Digital Object Identifier no. 10.1109/TC.2022.3218991

0018-9340 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

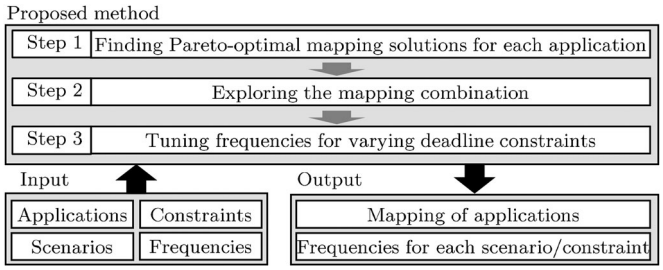


Fig. 1. Overall flow of the proposed mapping methodology.

the analysis and mapping exploration. To tackle the third challenge, the proposed method performs mapping in two phases. In the first phase, we find the Pareto-optimal mappings of *each* application based on the *direct* measurement on the real hardware platform. In the second phase, we explore the mapping combination of multiple applications based on the measured execution times obtained from the previous phase.

The fourth challenge comes from the restrictions imposed by the device and its SDK. TensorRT optimizes the network, saves it as an engine, and loads the engine to the assigned processor. Since the loading takes a long, a few seconds, it may not be done at run time without deadline violation. It means that dynamic task migration is not allowed. And the maximum number of DLA-mapped parts is limited. In addition, since there are only two levels of priority in GPU and no priority level in DLA, the priority assignment is usually not used for GPU and DLAs. Note that the traditional fixed priority-based worst-case response time (WCRT) analysis may incur over-estimated results in case many tasks have the same priority. Hence the mapping techniques based on the assumption that tasks are assigned different priorities are not suitable to our problem.

To tackle these challenges, we propose a novel scenario-based mapping methodology to map multiple DL applications onto heterogeneous processors. The overview of the proposed method is described in Fig. 1. It starts with the following input information: all DL applications to run in the given hardware platform, scenario information that specifies the applications to be run for each scenario, and various constraints to consider, such as real-time requirements of applications. In the first step, we obtain a set of Pareto-optimal mappings of each application onto heterogeneous processors in a pipelined fashion via an evolutionary meta-heuristic. The end-to-end latency and the average power consumption for each processor are objectives of mapping exploration in this step. For each Pareto solution, we partition the network of each application into sub-networks that will be converted to an engine to run on the mapped processor by TensorRT.

In the second step, we explore the possible mapping combinations of multiple applications considering the dynamic variation of scenarios via another meta-heuristic. Since no task migration is allowed, we should make the partitioned sub-networks of all networks resident for all scenarios. This step aims to minimize the average energy consumption for all scenarios while satisfying the schedulability condition on all processors. The schedulability analysis is performed considering the preemptive execution on the CPU and the FIFO execution on the GPU and DLAs. We explore the frequencies together since the frequency affects the execution time and energy consumption as well as schedulability.

As the last step, we adjust the frequencies of processors with a heuristic to save energy. Since the deadline constraint may vary at run time, the proposed scheme tunes the frequencies of processors for each deadline constraint and each scenario. For example, if the deadline constraint is alleviated or the transition to a less tighten scenario occurs, the frequencies can be lowered while satisfying the constraints.

As a result of the proposed mapping process, we get the static mappings for applications and frequencies for each scenario and each constraint. At run time, the frequencies are adjusted following the scenario and the associated constraints based on the resident mapping result. The proposed method is verified with extensive experiments using real-life applications, various scenarios, and real board.

Three main contributions of this paper can be summarized as follows:

- This paper reveals the technical challenges involved in multiple DL applications on a real embedded device that consists of heterogeneous processors, including NPUs.
- We propose a novel methodology for mapping multiple DL applications onto heterogeneous processors, tackling the technical challenges. While previous studies assume that the execution time of a layer on each PE is known before a mapping decision is made, such per-layer profiling is not possible for DLAs. And there are several implications imposed by the SDK that needs to be considered in making a mapping decision. The proposed methodology considers such limitations imposed by DLA and its SDK.
- While mapping decision is made statically, we adjust the frequency of processors dynamically to minimize energy consumption while satisfying the real-time constraints of all DL applications.
- The experiments are carried out on the real platform to show the efficacy of the proposed approach.

The rest of this paper is organized as follows. In the next section, we review the related work. Then, in Section 3, we explain the background. Section 4 presents a motivational example, notation, and problem definition. The proposed scheme is explained in Section 5. After the experiment results are discussed in Sections 6 and 7 concludes the paper.

2 Related Work

This section reviews the related work in the following two subsections involved in the proposed method: mapping/scheduling multiple applications and running multiple DL applications.

2.1 Mapping and Scheduling Multiple Applications

Since the mapping and scheduling problem of multiple applications on multiple processing elements (PEs) is known to be NP-hard [6], numerous sub-optimal techniques have been proposed. They are classified into a design-time approach,

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Comparison With the Related Works of Running Multiple Deep Learning Applications in Terms of: Optimization Approach, Pipelining, Use of NPU, Schedulability Check, Experiment on a Real Platform, and Energy Awareness

Works	Opt.	Pipe.	Using NPU	Sched. check	Real exp.	Energy aware
NestDNN [20]	Hybrid				✓	✓
DeepEye [21]	Hybrid	✓			✓	✓
S^3DNN [1]	Run			△	✓	
DART [2]	Design	✓		✓	✓	
LaLaRAND [4]	Run	✓		✓	✓	
Pujol et al. [22]	Design		✓	✓	✓	
Kang et al. [3]	Design	✓	✓	✓		✓
Proposed	Hybrid	✓	✓	✓	✓	✓

run-time approach, and a hybrid approach that combines the design-time analysis and run-time adaption [7].

In the design-time approach, it is assumed that all applications are running concurrently. If there is a dynamic variation of workload, the worst-case scenario is considered. Earlier works used list scheduling heuristics, named BIL [8] and HEFT [9], to schedule multiple applications onto heterogeneous processors. They assume that each application is specified by a DAG, and the execution time of a node on each processor is known.

Kang et al. [10] proposed a two-phase optimization scheme for the mappings of multiple applications represented by SDF (Synchronous Dataflow) graphs [11]. They find a Pareto-optimal set of static schedules for each graph via a genetic algorithm in the first step, then explore the combination of per-graph schedules with the schedulability analysis with another genetic algorithm. Their two-step approach is similar to our proposed technique, while their assumed hardware platforms are homogeneous processors.

If the workload varies at run time, the run-time approach is usually taken, aiming to maximize the load balancing if there are no real-time constraints. Niknafs et al. [12] proposed a mapping and scheduling method for incoming tasks. They optimize the energy consumption with the mixed-integer linear programming formulation and the heuristic while satisfying the deadline of tasks. Also, they investigated the effect of the workload prediction in terms of keeping the deadline and minimizing the energy consumption while not discussing how workload prediction can be made. Khasanov et al. [13] presented a run-time management method for multi-threaded applications to reduce energy consumption without deadline violation on a homogeneous multiprocessor system. They introduced the multi-dimensional knapsack problem-based heuristic for fast scheduling. Run-time techniques usually assume that the execution time of each application is known as a priori. Donyanavard et al. [14] collect data through periodic sensing and predict the performance and power based on the data. And they introduced a fast heuristic of run time task allocation to reduce the energy consumption while retaining the throughput. However, they did not consider real-time constraints.

To utilize the benefits of design-time approaches and run-time approaches, hybrid methods have been studied extensively. Some researchers assume that the dynamic behavior of a system can be modeled as a finite set of scenarios and make a static schedule for each scenario. Gheorgita et al. [15] introduced the concept of system scenarios and proposed the methodology to reduce the energy consumption. They identified the scenario and optimized the system per scenario at design time. At the run time, the scenario is predicted, and the system is arranged with a pre-optimized setting. Schor et al. [16] proposed a design flow for mapping applications on manycore systems representing the dynamic scenario variation with an FSM which has a finite number of scenarios. Assuming that the mappings are resident, they found a sub-optimal mapping for each scenario by an evolutionary algorithm. It aims to minimize the maximum core utilization.

The other researchers find a set of static mappings for each application and find an appropriate static mapping at run time depending on the scenario. Jung et al. [17] specified the dynamic behavior of applications by the dataflow model and FSM. They performed design-time analysis for each application subject to the number of processors. At run time, the number of used processors is changed according to the state to satisfy the throughput constraint. Quan et al. [18] proposed a scenario-based run-time mapping algorithm on an MPSoC-based embedded system. After finding a set of mappings of each application, they cluster workload scenarios and find the static mapping for each scenario. Their method finds the critical task that hinders achieving the goal throughput of each application and re-maps the task to fulfill the objective. Also, they expand their approach by adding a run-time throttling step that estimates the overhead of reconfiguration [19]. However, they evaluated the proposed technique with an in-house system-level simulator, not a real hardware platform.

To the best of our knowledge, there is no previous work that considered all technical challenges identified in this work. Our proposed technique belongs to the hybrid approach based on the finite set of scenarios similar to [16].

2.2 Running Multiple Deep Learning Applications

Research on the execution of multiple DL applications in embedded systems has recently become active. Table 1 summarizes the comparison of some related works and positions the proposed method.

NestDNN [20] presented a framework that schedules multiple DL applications considering run-time resource requirements. At design time, they created a multi-capacity model

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

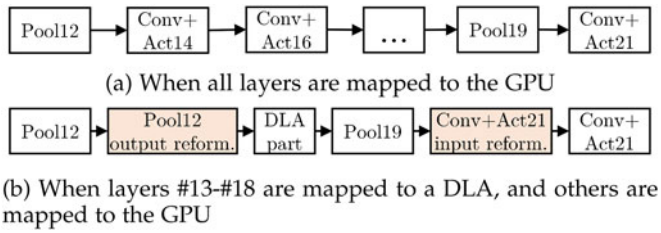


Fig. 2. The example composition of kernels.

consisting of models that provide a resource-accuracy trade-off for each application. Then, they monitor the resource and select the proper model for each application at run time. However, they did not exploit model-parallelism nor pipelining of each application. DeepEye [21] proposed a wearable camera that is capable of running multiple DL applications. They segregated layers into the computation- and memory-heavy layers at design time. At run time, they interleaved the execution of layers of multiple applications on a CPU-GPU heterogeneous system in a pipeline manner. It aims to reduce the latency of inference while prolonging the battery lifetime. Although both hybrid schemes, NestDNN [20] and DeepEye [21], considered the trade-off between energy and performance, they did not consider the real-time requirements of applications.

S^3DNN [1] introduced a run-time methodology that optimizes real-time correctness while increasing the throughput for multiple deep neural networks (DNNs). Their method selectively fuses multiple images to use fewer DNN instances and schedule instances to GPUs with a least-slack-first (LSF) policy. In spite of using the deadline-aware scheduling policy, it does not guarantee to keep the deadline. So, we put a triangle in the table to mean the partial consideration of the schedulability. They did not exploit the model parallelism of applications. DART [2] proposed a framework for multiple DNNs on a CPU-GPU heterogeneous system. They mapped the layers of DNNs onto CPU and GPU via a heuristic at design time. They ensured the schedulability of real-time applications and increased the throughput of best-effort applications. Albeit they applied the admission control for new tasks at run time, it does not change the mappings. LaLaRand [4] is a framework targeting a CPU-GPU heterogeneous system similar to DART. It allocates layers to processors at run time, supporting dynamic layer-level quantization. The main goal is to improve the schedulability of the system while minimizing the accuracy loss by layer-level mapping and quantization. All these methods [21], [2], and [4] exploit the temporal parallelism of inference by pipelining in a CPU-GPU heterogeneous system. However, their target platform does not contain an NPU that incurs several limitations in its usage.

Pujol et al. [22] and Kang et al. [3] presented the design-time method on a heterogeneous platform including NPU. The former implemented DL applications in Apollo [23] on the Xavier board. They executed multiple networks while keeping the schedulability. However, each neural network instance runs on a single type of processor in their method without exploiting the model parallelism or pipelining. The latter, Kang et al. [3], uses a GA technique to find per-layer mapping of multiple DL applications, aiming at minimizing the WCRT for each application and energy consumption.

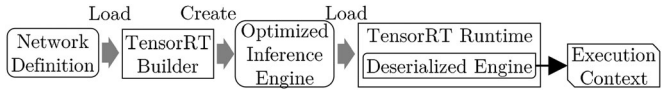


Fig. 3. The workflow of generating execution context in TensorRT.

They assume that layer-wise profiling information is given for all processing elements. Since it is not possible for NPU, they estimated the execution time of a layer on NPU by multiplying the GPU execution time and the average performance ratio between GPU and NPU. Unlike the proposed technique, they do not consider the scenarios, assuming that all DL applications run concurrently. Moreover, NPU is not used on a real platform in their experiments even though they considered the NPU in the analysis. With these limitations, they could find the mapping of all applications together at once. On the contrary, we propose a three-step approach to consider scenarios and the technical challenges imposed by the NPU and its SDK.

As displayed in Table 1, the mapping problem addressed in this work has not been tackled by the previous work, to the best of our knowledge.

3 Background

In this section, we review the characteristics of NVIDIA Jetson AGX Xavier (Xavier), the target platform for evaluation of the proposed methodology, and its SDK, TensorRT.

3.1 NVIDIA Jetson AGX Xavier

The Xavier equips three types of processors: an octa-core ARMv8 CPU, a single Volta GPU, and two NVIDIA DLAs. The DLA shows a power efficiency, but it is slower than GPU, and it does not support all types of layers. For example, some layers, such as *Yolo* layer, cannot be executed on DLA. Also, per-kernel profiling is not supported for DLA. The unit of profiling for DLA is a pipeline stage, which will be explained in the following section. The board has a unified and shared DRAM for CPU and GPU. It means that on-chip communication between CPU and GPU is carried out by DRAM memory access. However, there is an overhead in a DLA to load the data from DRAM to internal memory for the inference. It requires adding extra kernels to be executed for communication. For example, Fig. 2a indicates the kernel composition when running *Yolov2* network on the GPU. Fig. 2b shows the kernel composition when layers #13 to #18 are mapped to a DLA, and the rest are mapped to GPU. The colored *reformatter* kernel (input/output reform.) is added at the interface between the GPU and the DLA, as displayed in Fig. 2b. Since the overhead of the added kernel depends on the mapping, it needs to be considered in the mapping step.

3.2 NVIDIA TensorRT

TensorRT [5] is an SDK for high-performance inference targeted on NVIDIA products. Fig. 3 displays the workflow of TensorRT [24]. The optimized inference engine is generated by the *Builder* module from a given network definition. In the building process, some optimization techniques such as layer fusion are applied. Then the *Runtime* module loads and deserializes the engine to create execution context. The

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

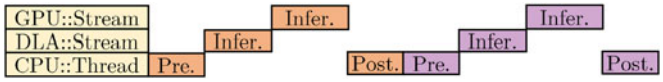


Fig. 4. An example schedule of deep learning inference.

Runtime module also maps a PE to the deserialized engine. The execution context is assigned to a stream in a GPU or a DLA. It is possible to run the network across multiple PEs in a pipelined manner by dividing a network into multiple pipeline stages. Each pipeline stage needs to be defined by a separate network with which a separate engine and the associated execution context are generated.

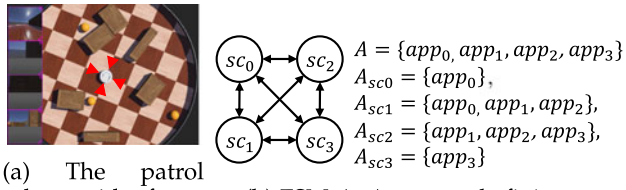
Fig. 4 illustrates an example schedule corresponding to the TensorRT implementation, where a single execution context is mapped to a DLA stream or a GPU stream. TensorRT does not support a CPU to execute the inference. The CPU performs pre/post-processing, marked as *Pre.* and *Post.* in Fig. 4. The pre-processing step loads the input data and adjusts the data layout for inference, while the post-processing step performs other processing after inference is done. Due to the execution environment of the Xavier board, the number of deserialized engines mapped to DLAs is limited to four at most at the time of writing this paper. It means that the total number of pipeline stages mapped to two DLAs may not be greater than four.

4 System Model

4.1 Motivational Example

Let us consider a patrol robot that explores a certain space, as shown in Fig. 5a. The robot moves around and searches for objects around it. The robot has cameras on all four sides: back and forth, left and right. It executes multiple object detection networks to detect objects while moving. We assume that objects within a certain distance can be detected by other means such as ultrasonic sensors. If an object is detected, we run an object detection network to identify the object. It means that the number of object networks may vary at run time. Such system behavior can be described by an FSM, as illustrated in Fig. 5b. Each state has the applications to be executed. In the figure, each application app_0, app_1, app_2, and app_3 denotes the application corresponding to each camera at the front, left, right, and back, respectively. For instance, state sc_1 indicates the state that consists of applications app_0, app_1, and app_2 running at the same time. In addition, each inference has a deadline because its result is used as an input to determine the action of the robot. Furthermore, the deadline constraint can vary depending on the speed of the robot. If the robot moves faster, then the object detection should be finished earlier. It is also important to reduce energy consumption since the robot is battery-powered.

In this paper, we propose a design methodology for running multiple DL applications with real-time constraints on a heterogeneous processor system while minimizing the energy consumption, to support this motivational example. Even though we use an FSM to describe the dynamic system behavior, what is needed in the proposed approach is how long the system stays and which applications are running in each state. Thus we omit the explanation of how state transition occurs and which state is the initial state. In this paper, we assume that the residence time of all states is the same for simplicity. If the state transition probability is given, however, we will be able to compute the relative residence time of all states. Since this computation is out of the scope of this work, it is left as future work.



(a) The patrol robot with front, rear, and side-view cameras

(b) FSM-A: An example finite state machine of the patrol robot

Fig. 5. Motivational example: patrol robot.

4.2 Notation

Notations used for system model and the problem definition are summarized in Table 2.

Architecture Specification: The target platform has a set of heterogeneous PEs, $\mathcal{PE}$, and processor type $proc$ can be one of CPU, GPU, and DLA. Each processor has a set of discrete frequencies: $freq_{CPU}$, $freq_{GPU}$, $freq_{DLA}$. We denote $|freq|$ as the sum of $|freq_{CPU}|$, $|freq_{GPU}|$, and $|freq_{DLA}|$.

Application Specification: We denote a given set of multiple DL applications as $\mathcal{A}$, and each application app_i consists of layers. After pipelining decision is made, an application app_i can be represented by a tuple $\langle S_i, p_i \rangle$, where S_i and p_i are the set of stages and the invocation period of app_i, respectively. Stage $s_j^i \in S_i$ is a pipelining stage of app_i corresponding to the pre- or post-processing step or an inference part partitioned to GPU or a DLA. The blue and green box in Fig. 6 illustrates two stages mapped to a DLA and the GPU, respectively. The applications run periodically with the implicit deadline assumption under which the period becomes the relative deadline.

TABLE 2
Notations for System Model and Problem Definition

Sign	Description
$\mathcal{PE}$	A set of heterogeneous PEs
$freq_{proc}$	A set of frequencies for processor type $proc$
$\|freq\|$	The sum of all $\|freq_{proc}\|$
$\mathcal{A}$	A set of multiple DL applications
app_i	An application in $\mathcal{A}$
S_i	A set of stages of app_i
p_i	A period and relative deadline of app_i
s_j^i	A jth stage of app_i
$\tau_k^{i,j}$	A kth task of s_j^i
$\|s_j^i\|$	The number of tasks in s_j^i
$P(s_j^i)$ and $P(\tau_k^{i,j})$	A period of s_j^i and $\tau_k^{i,j}$, respectively
$C_{s_j^i}$ and $C_{\tau_k^{i,j}}$	The WCET of s_j^i and $\tau_k^{i,j}$, respectively
SC	A set of scenarios
sc_i	A scenario in SC
$\mathcal{A}_{sc_i}$	A set of applications to be run on sc_i

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

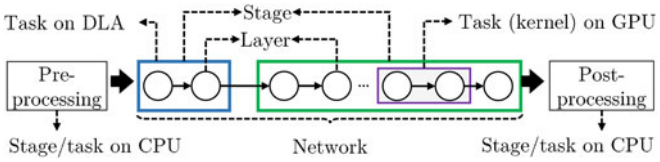


Fig. 6. Pipelining of the DL application.

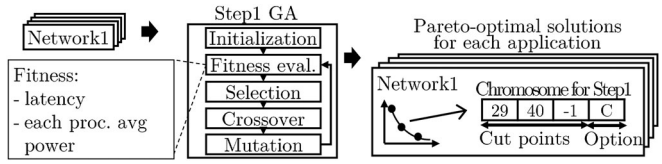


Fig. 7. The overview of step 1.

Mapping and Scheduling Specification: A task is a unit used in the analysis. Since the profiling granularity and the scheduling policy are different among processors, tasks are defined differently depending on the mapped processor. For GPU, a task corresponds to a kernel that may include one or more layers since TensorRT fuses layers to a single kernel via layer fusion, as displayed with a purple box in Fig. 6. While per-kernel profiling on the GPU is viable and each kernel runs in a non-preemptive way ([25], [26]), per-kernel profiling is not provided for DLA. Hence, the pipeline stage mapped to a DLA becomes a single task in the DLA (blue box in Fig. 6). The pre- or post-processing of the inference is a preemptable task that is mapped on the CPU. Stage s_j^i consists of one or more tasks, each of which is denoted by $\tau_k^{i,j}$, and the number of tasks in the stage is denoted by $|s_j^i|$. Note that the inference is sequentially executed due to the data dependency between tasks [2], and the order of task execution in the GPU is set by TensorRT [5]. Note that a DL application has a chain structure of tasks, which is also assumed in [2] and [4].

The period of a stage s_j^i or a task $\tau_k^{i,j}$, denoted by $P(s_j^i)$ or $P(\tau_k^{i,j})$ respectively, is the same as the period of an application, p_i. In addition, $C_{s_j^i}$ and $C_{\tau_k^{i,j}}$ represent the WCET of stage s_j^i and task $\tau_k^{i,j}$, respectively. We assume that the execution time of a task is inversely proportional to the frequency of the mapped processor. The mapping decision of app_i is denoted as a function $Map : S_i \rightarrow \mathcal{PE}$. A stage is a mapping unit, and it can be mapped to a single PE. The tasks in the same stage are mapped to the same PE. Data transfer overhead is considered since we measure the latency through direct measurement, including the extra kernels added by TensorRT as depicted in Fig. 2. Since the migration overhead is huge, the mapping decision is made statically and not changed at run time.

Scenario Specification: A set of scenarios is specified as an FSM where a state corresponds to a scenario. The set of states (scenarios) is represented as SC. Each scenario $sc \in SC$ consists of a set of applications to be run, A_{sc}, which is a subset of $\mathcal{A}$. For example, in Fig. 5b, A_{sc1} consists of three applications, app_0, app_1, and app_2.

4.3 Problem Definition

The problem addressed in this paper can be defined as follows.

Input: All applications in $\mathcal{A}$ and the scenarios, represented by an FSM, are given as input. The available frequency range of processors is already known at design time.

Constraints: The range of throughput constraints is given. For instance, it may be needed to process 15 or 30 image frames per second(FPS), depending on the robot speed. Since we assume the implicit deadline, the deadline constraint is inverse of the throughput constraint[1]. There exist other constraints imposed by the NPU and its SDK, as described in Section 1.

Objective: We aim to reduce the energy consumption of the system. If we do not know how long it stays in each scenario at runtime, it is not feasible to estimate the actual energy consumption. Thus, the average energy consumption over scenarios is used as the objective function in the experiments. Note that the proposed methodology can be applied with a different formula for expected energy consumption.

Output: The output of the design-time analysis is the mapping of stages in applications onto processors and the frequency of processors for each scenario and each deadline constraint. In a real deployment, the output includes the measured energy consumption.

5 Proposed Methodology

As shown in Fig. 1, the proposed methodology consists of three steps explained in this section.

5.1 Step 1: Finding Pareto-Optimal Mapping Solutions for Each Application

Because of the challenges imposed by the NPU and its SDK, we first determine the Pareto-optimal mappings for each application instead of making the mapping decision for all applications at once. We aim to simultaneously minimize the average end-to-end latency and average power consumption of each processor via a genetic algorithm (GA). Genetic algorithm is a widely used meta-heuristic inspired by evolutionary processes in nature, where sub-optimal solutions are found iteratively, starting with randomly generated initial solutions. A solution of the problem is encoded as a chromosome, and the objective function value, called *fitness*, of each solution is evaluated and compared in each generation. Since there exist several GA solvers publicly available, it is used in the current implementation of the proposed methodology. Other meta-heuristics can be used as long as multiple objectives are supported.

Fig. 7 illustrates the overview of this step. The GA gets the network as the input and initializes multiple chromosomes or generates a given number of initial random solutions. After fitness evaluation, we select a set of chromosomes and create new chromosomes (offspring) through evolutionary operations such as crossover and mutation. This step is performed on the real board since the fitness evaluation is

1. We use throughput constraint (FPS) and deadline constraint interchangeably to indicate a real-time constraint in this paper. For example, the throughput constraint of 30 FPS (frame per second) indicates the deadline constraints of 33,333us.

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Options for Intra-Network Pipelining

Options	# of pipeline stages	Composition of PEs
A	2	DLA0 - GPU
B	3	GPU - DLA0 - GPU
C	3	DLA0 - DLA1 - GPU
D	4	GPU - DLA0 - DLA1 - GPU
E	1	GPU

performed by direct measurement of latency and average power consumption on the real hardware platform.

The chromosome structure is shown on the right side of Fig. 7. The first three genes indicate pipeline cut points, and the last gene specifies the mapping option. A cut-point indicates the index of the last layer partitioned into a stage. For instance, the first pipeline stage consists of the 0th layer through the 29th layer. We compare five pipelining options that are listed in Table 3. Since the networks we used in this paper have some layers that cannot be performed on a DLA at the last part of the network, the last stage is assigned to GPU in all options. In options *C* and *D*, we distinguish two DLAs with *DLA0* and *DLA1*. In options *B* and *D*, the GPU is allocated two stages since the GPU has more computation power than DLA. In the example, option C is chosen in which three pipeline stages are mapped to DLA0, DLA1, and GPU, respectively, and two cut-points are specified: the second pipeline stage covers from the 30th layer to 40th layer, and the last pipeline stage starts from the 41st layer to the end of the network. A cut point value of -1 means that there is no corresponding cut point. The maximum number of cut-points is three for mapping option *D*. For mapping option *E*, no cut-point needs to be explored. It is noteworthy that DLA0 and DLA1 can be switched in the mapping options depicted in Table 3.

The proposed chromosome structure can be applied to other systems. For a given hardware platform, it is needed to define the possible mapping options first and increase the number of genes for pipeline cut-points up to the maximum number of pipeline stages in the mapping options.

The number of possible cut-point combinations depends on the number of layers and the number of pipeline stages. Since exhaustive exploration is very time-consuming, we reduce the number of cut-points to explore by filtering out some cut points. We exclude the cut points between the convolutional layer and its activation because the convolutional layer and the next activation layer are fused in the TensorRT. If there are consecutive layers that cannot be mapped to DLA, cut-points between them are also excluded.

For each Pareto-optimal solution, we estimate the WCET of each task on each processing element with a measure-based method. For the CPU task (pre/post-processing task), we measure the execution time using the POSIX time library. As for DLA, the assigned stage is regarded as a single task, and its execution time is measured by augmenting the code segment at the start and the end of the DLA task. As explained above, a GPU task corresponds to a kernel in the associated pipeline stage. The execution time of a task (kernel) on GPU is measured by the TensorRT IProfiler. To estimate the WCET of a task from measurements, we find the bigger value between the measured biggest value and the value obtained by adding six times the standard deviation to the mean. The estimated WCET of each task is saved in a database and used for the schedulability check in the next step. In addition, the average power of processors and average-case execution time (ACET) for each mapping solution are also measured. This information is used to calculate the fitness value in the next step. We use the maximum frequencies for all processors in this step.

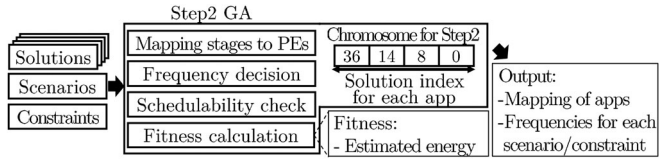


Fig. 8. The overview of step 2.

5.2 Step 2: Exploring the Mapping Combination

After Pareto-optimal mapping solutions for each application are determined, we find a sub-optimal combination of individual mapping solutions of all applications and clock frequencies of processors for scenarios. To find a sub-optimal combination, a GA and two heuristics are devised, aiming to minimize the expected energy consumption.

Algorithm 1. Pseudo Code for Chromosome Evaluation

```
1:  procedure EvaluateChromosomechromosome
2:    Check the number of DLA mapped stages
3:    if not executable on DLAs then
4:      return MaxInt
5:    mappings = MapStageToPE(chromosome)
6:    totalFitness = 0
7:    for each scenario do
8:          schedulable, fitness = SetFreq(scenario,mappings)
      ▷Schedulability check and fitness calculation are done in this
      function
9:          if not schedulable then
10:           return MaxInt
11:       totalFitness += fitness
12:   return totalFitness / the number of scenarios
```

The workflow of this step is described in Fig. 8, and the chromosome structure of the GA is shown in the center of the figure. Each gene represents a mapping solution index. In the example of Fig. 8, the mapping of the first application is the 36th mapping among all Pareto-optimal mappings of the corresponding application. Since we cannot change the mapping of an application dynamically due to the limitation of the SDK, the selected mapping defined by a gene is fixed in all scenarios.

How to compute the fitness value of a chromosome is depicted in Algorithm 1. First, it checks whether the number of tasks (stages) assigned to DLAs is more than four (line 2). If it exceeds four, the maximum fitness value is assigned to indicate that this chromosome cannot be a feasible solution. For a feasible solution, mapping stages to PEs is determined by a heuristic (line 5). Note that the mapping solution of an application does not say which DLA is used between two

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

DLAs. Thus we devise a simple heuristic to determine which DLA will be used for each application.

Algorithm 2. Pseudo Code for Mapping Stages to PE

```
Require AppList: the list of applications ▷In the decreasing order of the max length of DLA stage (task)
        Require mappings: the table to save mapping info
 1: procedure MapStageToPE(chromosome)
 2:   CPUIdx, DLAIdx = 0, 0
 3:   Parse mapping option of app                ▷Based on chromosome
 4:   nearAppList = MakeNeighborAppList()
 5:   for each app in AppList do
 6:     if app is already mapped then
 7:       continue
 8:     for each stage do
 9:       if stage maps on CPU then
10:         mappings[app][stage] = CPUIdx
11:         CPUIdx = (CPUIdx + 1) % CPUCoreNum
12:       else if stage maps on GPU then
13:         mappings[app][stage] = 0    ▷One GPU on the board
14:       else stage maps on DLA
15:         mappings[app][stage] = DLAIdx
16:         DLAIdx = (DLAIdx + 1) % DLANum
17:     for nearApp in nearAppList[App] do
18:       Repeat the line 6 to 16 for nearApp
19:   return mappings
```

Algorithm 3. Pseudo Code for Frequencies Decision

```
Require ProcList: the list of processors ▷In the order of GPU, CPU, and DLA
 1: procedure SetFreq(scen,maps)
 2:   set frequencies for all processors (freqs) as maximum
 3:   i, schedulable, minSchedulable, minFitness = 0, False, False, MaxInt
 4:   Save curr freqs as the best freqs
 5:   for two iterations do
 6:     for each processor in ProcList do
 7:       set curr processor frequency as maximum
 8:       while True do
 9:         schedulable = checkScheduability(scen,maps,freqs)
10:         if schedulable then
11:           fitness = calculateFitness(scen,maps,freqs)
12:           if fitness < minFitness then
13:             save curr freqs as the best freqs
14:             minFitness, minSchedulable = fitness, True
15:         else
16:           if curr frequency is not the maximum then
17:             Revert to the previous frequency
18:           break
19:         if curr frequency is the minimum frequency then
20:           break
21:         else
22:           Set curr processor frequency one level down
23:   return minSchedulable, minFitness
```

Next, the schedulability test for each scenario is performed on each processor. How to perform the schedulability test will be explained below in this section. Since the clock frequency affects the execution time of each task on a mapped processor, we select the frequencies of processors to minimize the energy consumption for each scenario while satisfying the schedulability constraint (line 8). If no feasible schedule is found in any scenario, the maximum fitness value is immediately returned to mark the solution as infeasible. Otherwise, the average fitness value over all scenarios is returned.

5.2.1 Mapping Stages to PEs

The proposed heuristic for mapping stages to PEs is depicted in Algorithm 2. At first, it parses the mapping option for each selected mapping of application (line 3). Then, the neighbor application list is made for each application (line 4). The list of neighbor applications of the target application consists of the applications that will be executed simultaneously in some scenarios. We sort the applications in the decreasing order of the maximum length of the DLA stage (task). Mapping is done in a round-robin fashion using the mapping index on each processor (lines 5-16). If a stage is mapped to the CPU, it is mapped to the indexed core of the CPU, and the index is incremented. In the case of GPU, the mapping index is always zero since there is only one GPU in the target board. If a stage is mapped to a DLA, two DLAs are alternatively mapped. Since the applications are sorted based on the DLA workload, the heuristic aims to balance the workloads of two DLAs available in the hardware platform. After the mapping for the current application is complete, we map the neighbor applications in the same fashion (lines 17-18).

5.2.2 Frequency Decision

After the mapping of stages is settled, we determine the frequency of each processor. Since the frequency affects the execution time, we perform the schedulability test for each frequency combination. Since the number of frequency combinations is too large to be explored exhaustively inside a GA, a heuristic is devised to explore the frequencies in a greedy fashion as depicted in Algorithm 3. The initial frequency combination is set by using the maximum frequency of each processor (line 2). Since the algorithm reduces frequencies in a greedy way, we examine the reduction amount of power consumption per each frequency change and determine the order based on the reduction amount. The order turns out to be GPU, CPU, and DLA in the process of fitness calculation covered in the next paragraph. We select the processor whose frequency is to be changed (line 6), starting from the maximum frequency (line 7), and check the schedulability with the current frequency combination (line 9). If it passes the schedulability test, we calculate the fitness value that is the expected energy consumption and save it if it is the minimum fitness (lines 11-14). And we reduce the current processor's frequency one level down (line 22). If the schedulability test fails, we go back to the previous frequency (line 17). We repeat this process until the frequency becomes the lowest one (lines 19-20). Then, we select the next processor whose frequency is to be altered (line 6), starting from the last schedulable frequency combination found so far. In the first iteration, we find the lowest schedulable frequency in the order of GPU, CPU, and DLA in a greedy fashion. In the next iteration, it searches the frequency range between the highest frequency

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

and the found frequency to check if the energy can be lowered with a higher frequency, again in a greedy fashion.

5.2.3 Fitness Calculation

Fitness is defined as the average energy consumption for all scenarios in a given period. We use the average value for all scenarios since we do not know which scenario to stay how long. It can be applied differently if we have the probability information. To estimate energy consumption, we first identify the relationship between the frequency and power consumption for each processor by the regression analysis based on the measurement. We consider both static and dynamic power consumption.

For the estimation of static power consumption, we change the frequencies over an available range as displayed in Table 7[2]. We estimate the static power on a fixed frequency when no application runs, which is similar to the work of [28][3]. So we figure out the relation between frequency and static power by measuring the power after setting the minimum and maximum frequencies to be equal.

For dynamic power estimation, we measure the average power consumption of each application by varying the frequency of each processor. For the CPU and GPU, we use the mapping option *E*. On the other hand, option *A* is used for DLAs with all mappable layers for DLA. When changing the frequency of one processor, we set the frequencies of the other processors to be maximum. Then, we obtain the relationship between the frequency and dynamic power by subtracting the estimated static power from the measured total power. Such regression analysis is conducted individually for four benchmark applications that are described in Table 4.

With the obtained relationship between the frequency and dynamic power, we estimate the average dynamic power consumption of the given mapping at a given frequency. The average power consumption of each processor on each mapping solution is measured in step 1 with the maximum frequencies as explained in Section 5.1. We apply the relationship to estimate the dynamic power consumption of processors at the given frequencies for a given mapping. The dynamic energy consumption is computed by multiplying the computed dynamic power consumption and the average execution time. The average execution time is estimated to be inversely proportional to the frequency from the profiled ACET. For static energy consumption, we multiply the estimated static power consumption and the period. The estimated energy consumption is the sum of the static and dynamic energy consumption.

5.2.4 Schedulability Analysis

For schedulability analysis, we estimate the worst-case response time (WCRT) for each chain-structured DL application by tailoring the compositional performance analysis [29]. If the sum of the WCRTs for each part of an application considering the jitter is no greater than the deadline constraint, the deadline constraint is satisfied. We can assign the distinct priority levels to the tasks mapped on the CPU, and the tasks are scheduled by preemptive scheduling with a fixed priority. However, the tasks mapped on GPU or DLA have the same priority.

TABLE 4
The Benchmark Networks and the Volume of Design Space for Step 1 to Find the Pareto-Optimal Mappings

Network	Layer #	Selected cut-points #	Total cand. #	Pareto sol. #
Yolov2 [35]	54	22	2025	112
Yolov3 [36]	179	78	82161	158
Yolov2tiny [35]	24	10	221	19
Yolov3tiny [36]	35	10	221	14

The worst-case response time of a CPU task, $\tau_k^{i,j} \in s_j^i$ of app_i, is bounded by equation (1) which is well-known for fixed-priority preemptive scheduling ([29], [30]). The WCRT of the task is the converged value of r^m. The $hp(\tau_k^{i,j})$ is a set of higher or equal priority tasks that are mapped to the same PE with $\tau_k^{i,j}$ and belong to the other applications. For simple notation, τ_h indicates a task in $hp(\tau_k^{i,j})$. J_{τ_h} is the jitter of the τ_h.

$$r^{m+1} = C_{\tau_k^{i,j}} + \sum_{\tau_h \in hp(\tau_k^{i,j})} \left\lceil \frac{r^m + J_{\tau_h}}{P(\tau_h)} \right\rceil \cdot C_{\tau_h} \quad where\ r^0 = C_{\tau_k^{i,j}} \tag{1}$$

In the case of GPU, the kernels within a stream are executed in a FIFO order, and kernels are executed in a non-preemptive way ([25], [26]). Since the priority of all GPU tasks is equal, the WCRT may be over-estimated if we consider worst-case interference from all tasks. Thanks to the chain structure of an deep learning application and the FIFO scheduling policy of GPU, we can reduce the pessimism of the analysis. The worst-case response time of stages mapped to the GPU can be calculated by the formula as shown in equation (2).

$$r = C_{s_j^i} + \sum_{s_e \in ep(s_j^i)} B_{s_j^i, s_e} \tag{2}$$

In this formula, $ep(s_j^i)$ means a set of stages that are mapped to the same PE with s_j^i of app_i and belong to the other applications. $B_{s_j^i, s_e}$ represents the maximum interference from stage s_e to s_j^i where s_e indicates a stage in $ep(s_j^i)$. It is the maximum sum of WCETs of as many successive tasks in s_e as $min(|s_j^i|, (\lceil \frac{P(s_j^i)}{P(s_e)} \rceil + 1) \cdot |s_e|)$, which is likely to be smaller than the entire WCET of stage s_e, C_{s_e}. Suppose that stage s_j^i consists of 7 kernels (tasks), and the interfering stage s_e consists of 3 kernels. Since kernels in a stage are queued sequentially, a kernel of an application can be interfered by at most one kernel of the other application, thanks to the FIFO scheduling policy. Thus the maximum number of interfering kernels cannot be more than the number of

2. While the Xavier board allows a user to change the processor frequencies [27], the voltage is under the direct control of the board power management firmware. Hence, we tune the processor frequencies only.

3. The static power in this paper means the power when no application runs in the P-state. So, the static power can be affected by the frequency.

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

kernels in s_j^i, $|s_j^i|$. In case the number of kernels in the interfering stage is smaller than $|s_j^i|$, we compute the maximum possible number of interfering kernels by multiplying the number of kernels in the stage and the number of interference, which is bounded by $\lceil \frac{P(s_j^i)}{P(s_e)} \rceil + 1$. If stage s_e that has three kernels can interrupt twice maximally, the six kernels may interfere the target stage s_j^i at most. Then, the minimum value between $|s_j^i|$ $(= 7)$, and $2 \cdot |s_e|$ $(= 6)$ becomes the number of kernels that may interfere.

Since the stage mapped to a DLA consists of a single task as explained in Section 4, the schedulability test for a DLA also can be performed with equation (2). The estimated worst-case response time of an application is the sum of the response time of the stages considering the jitter. Since we assume implicit deadline constraints, the estimated WCRT should be no greater than the period for each application.

5.2.5 Complexity Analysis

Since Algorithm 1 calls Algorithms 2 and 3, we first investigate the complexity of Algorithms 2 and 3. First of all, we assume that the schedulability analysis has a given time complexity, $T(\mathcal{PE}, \mathcal{A}_{sc_i})$, which is a function of processors and applications. Even though the response time analysis is known as NP-Hard [31], it is usually computed fast for the problem size assumed in this paper. In Algorithm 2, the amount of computation is proportional to the number of all stages because the mapping is determined in a round-robin manner so that its time complexity is $O(\Sigma_i |S_i|)$. Since it uses a space for $nearAppList$, the space complexity is $O(|\mathcal{A}| \cdot |\mathcal{A}| + \Sigma_i |S_i|)$.

Algorithm 3 has a nested loop of two levels. Since each loop runs at most the summation of the number of frequencies for each processor, the time complexity is $O(|freqs| \cdot T(\mathcal{PE}, \mathcal{A}_{sc_i}))$. The space complexity of Algorithm 3 is $O(|freq| + \Sigma_i |S_i| + |\mathcal{PE}|)$, including the space for frequencies and processors. Since Algorithm 1 calls Algorithms 2 and 3 for each scenario, Algorithm 1 has the time complexity of $O(|SC| \cdot (\Sigma_i |S_i| + |freqs| \cdot T(\mathcal{PE}, \mathcal{A}_{sc})))$ and the space complexity of $O(|freq| + \Sigma_i |S_i| + |\mathcal{PE}| + |\mathcal{A}| \cdot |\mathcal{A}|)$.

5.3 Step 3: Tuning Frequencies for Varying Deadline Constraints

While we consider the tightest deadline constraint in the previous step, the deadline constraint can be loosened at run time if the robot moves slowly. It means that there is a chance to use lower frequencies while satisfying the deadline constraint when the deadline constraint is loosened. In the last step of the proposed methodology, we explore the frequency combinations, varying the deadline constraints. We use the same heuristic as Algorithm 3 for each deadline constraint. Since running the heuristic at run time incurs non-negligible overhead, we sample the deadline constraints and construct a table that contains the frequency combination for the sample deadline constraint. For example, if the maximum deadline constraint is given as 30 FPS[4], we define a set of discrete deadline constraints and perform the heuristic at design time. At run time, we refer to the table to get the frequency combination of the closest tighter deadline constraint than the actually required deadline.

4. Since we assume the implicit deadline, the deadline constraint is inverse of the throughput constraint. The higher FPS indicates the tighter deadline constraint.

6 Experiments

6.1 Experiment Set-Up

The hardware platform used in the experiments is a Jetson AGX Xavier board with Jetpack 4.3 and TensorRT 6. We use the tkDNN [32] library and JEDI [24] framework to deploy DL applications in a pipeline way on TensorRT easily. We use four deep learning networks described in Table 4 as benchmark applications that are specified with tkDNN and JEDI. Since the *leaky relu* activation is not supported on the DLA, we used the networks, provided in JEDI, that replace *leaky relu* with *relu*. The *COCO2017 val* is used for inference with an image size of 256x256 with FP16 precision, and the batch size is set to one. In the target board, the power consumption is measured by the *tegrastats* command.

For GA implementation, the DEAP [33] framework is used. In the first step of the proposed method, the maximum evolution count is set to 20,000 and the population size to 65. In each generation, two chromosomes are created as offspring of each survivor through genetic operations such as uniform crossover, one-point mutation, and random selection. We execute the GA on the target board since it requires running the network to obtain the performance value by measurement. In the second step, we proceed up to 10,000 generations with 128 chromosomes in the population. The same operations in the previous step are used except that Lexicase [34] is used for the selection operation. To check the maximum achievable deadline constraint, we find the mappings from the lower constraint, then use the found mapping as an initial solution for exploring the higher constraints. In this step, the GA is run on a host computer that consists of Ubuntu 18.04.6 LTS, AMD Ryzen 9 3950X 16-Core Processor with 64GB RAM.

6.2 Finding Pareto-Optimal Mappings of Each Application

As mentioned in Section 5.1, we filter out the cut points to reduce the design space. Table 4 shows the number of layers and selected cut points for each network. The number of candidate cut points is reduced to fewer than a half of the total number of layers. The total number of cut-point combinations is calculated based on the mapping options of Table 3. For example, in the case of *Yolov2*, it is calculated as follows: $\binom{22}{1} + \binom{22}{2} \cdot 2 + \binom{22}{3} + \binom{22}{0} = 2025$. The last column shows how many Pareto-optimal solutions could be obtained by the proposed GA. The number of Pareto-optimal solutions is $0.2\% \sim 8.6\%$ of total candidates.

6.3 Exploring Mapping Combination and Tuning Frequencies

In this experiment, we evaluate the second step of the proposed scheme with four different system behaviors that are represented by FSMs. In addition to our motivational example of Fig. 5b denoted by *FSM-A*, Table 5 shows which applications are performed in each state for other three FSMs. Since the transition of states does not affect the result

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

TABLE 5
Information on Three Different System Behaviors: Which Applications are Performed in Each State

(a) FSM-B

State	Applications ($\mathcal{A}_{sc}$)
sc_0	app_0
sc_1	app_3
sc_2	app_0, app_1
sc_3	app_1, app_3
sc_4	app_0, app_2
sc_5	app_2, app_3
sc_6	app_0, app_1, app_2
sc_7	app_1, app_2, app_3

(b) FSM-C

State	Applications ($\mathcal{A}_{sc}$)
sc_0	app_0, app_3
sc_1	app_0
sc_2	app_3
sc_3	app_0, app_1
sc_4	app_1, app_3
sc_5	app_0, app_2
sc_6	app_2, app_3
sc_7	app_1, app_2

(c) FSM-D

State	Applications ($\mathcal{A}_{sc}$)
sc_0	app_0
sc_1	app_0, app_1
sc_2	app_0, app_1, app_2
sc_3	app_0, app_1, app_2, app_3

of the mapping, we omit the transition information between state.

FSM-B is an extended version of *FSM-A* with twice more scenarios. In *FSM-C*, the maximum number of applications running concurrently is limited to two. Meanwhile, the number of applications is increasing in *FSM-D*. The most important factor is the maximum number of applications running concurrently since the schedulability test is conducted in this step. We also conduct the experiments with two different combinations of applications as denoted by cases α and β shown in Table 6. In the case of α, all applications from app_0 to app_3 are the same network, *Yolov2*. We assign the priority of tasks mapped to CPU in the following order: app_0, app_3, app_1, app_2. On the other hand, the tasks mapped onto GPU or DLA have an equal priority. Although all applications are assumed to have the same period in all experiments, the proposed method allows them to have distinct periods.

We compare the proposed method with the following three methods.

- Local(LC): It allows dynamic task migration across the scenarios. We find the sub-optimal mapping of applications for each scenario by a GA. However, it is not feasible in practice since dynamic task migration is too costly in the target platform. Nonetheless, this method serves as a decent indicator to check how close is the proposed method to the ideal case.
- Baseline(BS): It is a typical case of running each deep learning application on a single processing element, GPU. Since all applications have some layers that are not supported by DLA, it uses only GPU in the inference, which corresponds to option E in Table 3.
- Global(GB): It uses a GA to find a sub-optimal static mapping of all applications, assuming that all applications are running concurrently, ignoring the scenarios. It produces the same mapping for all system behaviors from *FSM-A* to *FSM-D*. The frequency for each processor is set to the maximum without applying step 3.
- Work of [3](SOTA): It also explores static mapping with maximum frequencies ignoring scenarios similar to GB, but it uses the estimated layer-wise execution times. After obtaining the mapping result from the SOTA method, we recalculate the fitness and schedulability analysis with the actual execution times for fair comparison.
- Proposed(PR): It finds a static mapping combination tailored for each system behavior. It explores the design space with the objective to reduce the average energy consumption by considering scenarios and constraints.

In this experiment, we make three comparisons. Firstly, we compare the tightest deadline constraint that each method can support without schedulability violation. Also, the comparison is made in terms of the fitness value. Then we evaluate the effect of frequency tuning in terms of energy reduction. Lastly, we confirm the necessity of the proposed heuristics compared with the exhaustive search in terms of speed.

TABLE 6
Cases for a Combination of Applications in FSMs

Case	app_0	app_1	app_2	app_3
α	Yolov2	Yolov2	Yolov2	Yolov2
β	Yolov2	Yolov2tiny	Yolov3tiny	Yolov3

6.3.1 Comparison of the Maximum Achievable FPS

Table 8 shows the maximum FPS that each method can achieve. In all FSMs except for *FSM-D*, the PR method can satisfy the tighter deadline constraint (higher FPS) than both BS and GB methods. The PR could satisfy $15\% \sim 86\%$ tighter constraint compared to the BS method, and up to 40% tighter constraint than the GB method. Since the static mapping is determined by the worst-case scenario in the PR method and *FSM-D* runs all applications in state *SC4*, PR and GB have the same FPS constraint for *FSM-D*. While comparison with BS confirms the advantage of utilizing heterogeneous processors, comparison with GB confirms the

TABLE 7
The Frequency Range Used in the Exploration

Processor	Frequency range (MHz)
DLA	550.4, 640, 729.6, 806.4, 896, 972.8, 1062.4, 1152, 1228.8, 1395.2
GPU	675.75, 828.75, 905.25, 1032.75, 1198.5, 1236.75, 1338.75, 1377
CPU	1190.4, 1267.2, 1344, 1420.8, 1497.6, 1574.4, 1651.2, 1728, 1804.8, 1881.6, 2035.2, 2112, 2188.8, 2265.6

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

TABLE 8
Comparison of the Maximum Achievable FPS Among Methods

Methods	Case α - FSMs				Case β - FSMs			
	A	B	C	D	A	B	C	D
LS	47	47	71	40	32	32	38	28
BS	32	32	44	24	21	21	33	15
GB	40				28			
SOTA [3]	39				26			
PR	47	47	56	40	32	32	38	28

advantage of considering the scenarios in the mapping decision. The work of [3] shows slightly worse results than the GB method. When using estimated time, the mappings obtained by [3] could satisfy 50 FPS and 32 FPS for each case, respectively, but recalculation with the actual execution times shows different results. This is because the worst-case execution time could be underestimated when using the estimated time so that an infeasible solution can be falsely classified as feasible.

The proposed method shows a similar result as LC except for *FSM-C* of case α. In the LC method, DLA mapping can be changed for each scenario. Hence we can assign a different DLA to each application when two applications are running in all scenarios in *FSM-C*. On the other hand, two applications are mapped to the same DLA in some scenarios in the PR method. In case β, the entire *Yolov3* is mapped GPU since its execution time is much larger than the other networks and GPU is faster than DLAs. Since GPU becomes the bottleneck in the schedulability check, the PR yields the same result to the ideal one, LC. Comparison with LC proves the effectiveness of the proposed static mapping technique.

Table 9 a shows a mapping decision example that the proposed method produces for *FSM-A*. In the table, *D[0-40], G[41-53]* means that pipelining option A is taken, layers #0 to #40 are mapped to DLA, and the rest are mapped to GPU. In case α, all applications use the same mapping result since they are all *Yolov2*. In case β, however, the selected options are different for applications: option C is used for *app0* and option E is used for *app3* while option A is used for the others. It indicates that we need to explore different mapping options for applications. Table 9 b describes how frequencies can be changed as the deadline constraint varies at run-time after mapping decision is made to achieve the maximum FPS as shown in Table 9 a. The DLA frequency, which is not shown in the table, turns out to be maximum at all times. It means that reducing the GPU or the CPU frequency is better than reducing the DLA frequency in terms of average energy consumption. Since the GPU consumes more power than the CPU, the proposed heuristic tries to reduce the GPU frequency as much as possible while satisfying the schedulability constraint. Since the CPU performs pre-processing and post-processing tasks that have a linear dependency on the inference part, we may need to reduce the CPU execution time by increasing the CPU frequency under a lower FPS constraint in order to satisfy the constraint on the end-to-end worst-case response time. While the GPU frequency decreases as the FPS constraint is loosened, there is no such tendency in the CPU frequency. It explains why the CPU frequency increases as the FPS constraint decreases from 30 FPS to 25 FPS for case β in Table 9 b.

TABLE 10
Throughput Constraint (FPS) Variation

Case	FSM-A	FSM-B	FSM-C	FSM-D
α	35, 40, 45	35, 40, 45	40, 45, 50	30, 35, 40
β	20, 25, 30	20, 25, 30	25, 30, 35	15, 20, 25

To evaluate the effectiveness of the proposed GA-based heuristic for the second step of the proposed scheme, we find an optimal mapping of the GB method by exploring exhaustively the design space that is vast (112^4 mappings for the case α). It is observed that the resultant mapping is the same as found by the proposed GA heuristic. Since only a few mappings can satisfy the deadline constraint, the proposed GA-based scheme could find the optimal solution in our experiments, we believe. The proposed GA-based heuristic explored much less design space. This experiment confirms the efficacy of the proposed GA-based exploration method as the second step of the proposed methodology.

6.3.2 Frequency Tuning Effect

In this experiment, we examine the effect of frequency tuning as the deadline constraint varies at run time. For a given mapping, we visit each scenario to find the best frequency combination of processors in terms of estimated energy consumption. Note that we can obtain a higher FPS performance than that of Table 8

TABLE 9
Mapping and Frequency Tuning Result for FSM-A From the Proposed Method

(a) Mapping result for FSM-A

Case	FPS constraint	app_0	app_1	app_2	app_3
α	47	D[0-29],G[30-53]	D[0-29],G[30-53]	D[0-29],G[30-53]	D[0-29],G[30-53]
β	32	D[0-29],D[30-40],G[41-53]	D[0-16],G[17-23]	D[0-16],G[17-34]	G[0-178]

(b) Frequency (MHz) tuning example on SC3 of FSM-A for varying FPS constraints with the same static mapping as (a)

Processor	Case α			Case β		
	40 FPS	45 FPS	47 FPS	25 FPS	30 FPS	32 FPS
CPU	1574.4	2112.0	2265.6	1958.4	1651.2	1728.0
GPU	1032.75	1236.75	1338.75	1032.75	1338.75	1377.0

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

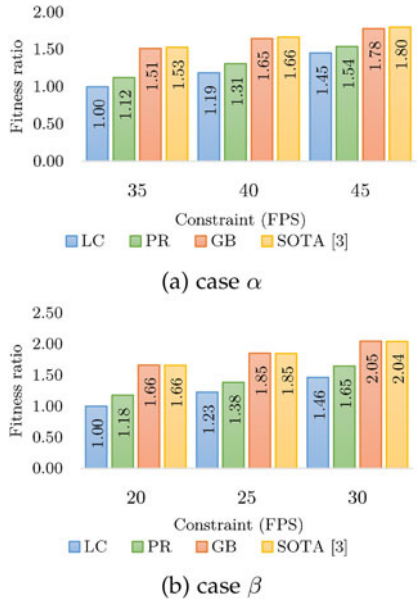


Fig. 9. Fitness ratio as the FPS constraint varies for FSM-A.

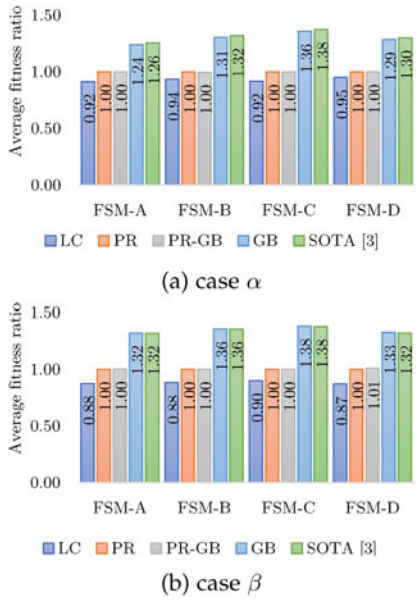


Fig. 10. Average fitness ratio over the FPS constraint variation for all benchmarks.

for the GB method in case there is no scenario that runs all applications. With the given mapping obtained from the GB method, we increase the throughput constraint until there is any scenario that cannot meet the constraint. The modified maximum achievable FPS from the GB method for each FSM turns out to be 45, 45, 51, and 40 FPS for case α, while it is the same as PR in Table 8 for case β. It is still lower than the maximum achievable FPS obtained from the PR method. We vary the deadline constraint as described in Table 10, where the tightest constraint is determined by the GB method.

Fig. 9 shows how the fitness ratio varies as the deadline constraint varies for the *FSM-A* benchmark with cases α and β, respectively. The fitness value represents the estimated energy consumption during the same time duration. The fitness ratio is the relative fitness over the lowest value. The figure demonstrates the energy can be reduced while satisfying the constraint by manipulating the frequencies. In addition, the difference between the PR and both GB and SOTA methods is large, while the gap between PR and LC methods is small. In this comparison, the GB method does not tune the frequency as the constraint varies.

To study the effectiveness of frequency tuning, we propose an alternative method which is a variant of the PR method. We apply the frequency tuning algorithm to the resultant mapping obtained by the GB method, calling this method as PR-GB. Fig. 10 shows the average fitness ratio of methods over the FPS constraint variation. In this figure, the fitness ratio is the relative fitness over the fitness value of the PR method on each bundle. While the difference between methods PR and GB is noticeable, PR and PR-GB show almost no difference. Also, GB and SOTA methods give similar results. It means that the frequency adjustment is more important than the mapping decision for energy reduction. In addition, the difference between the two proposed methods (PR and PR-GB methods) and the ideal case (LC method) is smaller than the gap between the proposed methods and both GB and SOTA methods.

6.3.3 Execution Time for Fitness Evaluation

Since frequency tuning is a key issue for energy reduction, we explore the frequency combination of processors in the fitness evaluation of each candidate mapping in step 2, as explained in Section 5.2.2. Table 11 shows the average execution time for fitness evaluation and the end-to-end time of the proposed method during 50 generations in the proposed GA-based heuristic, compared with a brute-force exhaustive method. The speedup by the heuristic is more than at least 26 times for fitness evaluation and 28 times for end-to-end execution than the brute-force method. It is confirmed by experiments that both schemes produce the same

TABLE 11
Average Fitness Evaluation Time and End-to-End Execution Time During 50 Generations in GA

Case	Method	Average fitness evaluation time (sec)				End-to-end execution time (sec)			
		FSM-A	FSM-B	FSM-C	FSM-D	FSM-A	FSM-B	FSM-C	FSM-D
α	heuristic	1.41	3.44	3.08	1.68	317.66	1119.19	1268.51	439.59
	brute-force	46.13	99.76	87.30	57.08	13390.05	35457.24	36095.17	17972.25
β	heuristic	2.28	5.07	3.98	3.30	519.03	1223.27	726.65	867.07
	brute-force	61.00	131.78	118.78	87.21	17405.57	34586.12	23861.59	27643.66

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

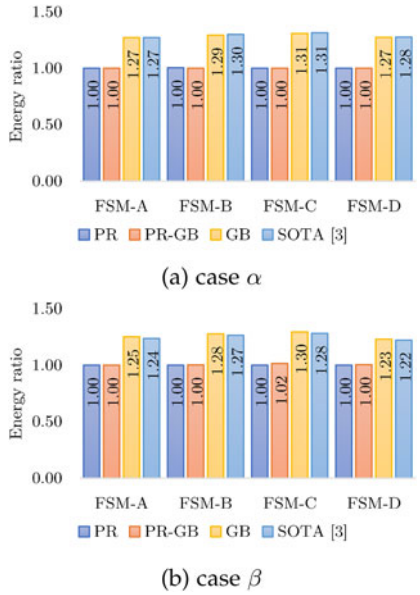


Fig. 11. Average energy ratio over the FPS constraint variation, measured after real deployment.

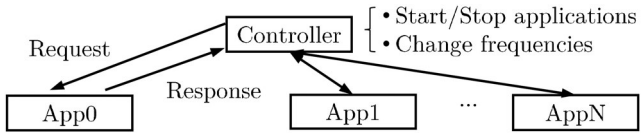


Fig. 12. The run time management following scenarios.

fitness value for the resultant mappings. It proves the effectiveness of the proposed frequency selection heuristic.

6.4 Real Deployment

We apply the proposed methodology to run benchmark applications on the target board. A run-time controller is implemented to control the execution status of applications according to the scenario, as illustrated in Fig. 12. It consists of multiple threads to communicate with the applications. In the experiment, a thread requests execution of the associated application to process each input periodically and suspends itself until the completion of the processing is informed. The communication overhead between the controller and applications is considered in the latency estimation at the design time by subtracting the overhead from the deadline at the design-time optimization. The communication overhead is set by the bigger value between the measured value and the value that the mean plus six times the standard deviation. The frequencies are managed by the controller for each scenario and each constraint. While we change the processor frequencies, we set the frequency of the other components to the maximum. The controller runs on a single dedicated core which is not used for DL applications. Note that the run-time management itself is not a contribution of this paper.

In this experiment, we run applications on the target board during the same duration, 10 seconds, varying the scenarios for each mapping and frequency tuning result for each deadline constraint. The energy consumption is calculated by multiplying the running time and the average power consumption of the system that includes not only processors but also the other components. We compare the PR, PR-GB, GB, and SOTA methods, excluding the LC method that is infeasible on a real platform. We perform each experiment three times and get the average value. Fig. 11 shows the measured average energy consumption over the FPS variation for two cases α and β. The energy ratio indicates the relative energy consumption over the lowest value on each bundle. In both cases, the gap between PR and PR-GB is not noticeable, as expected by our estimation result. The proposed methods could save the energy by $22\% \sim 31\%$, compared with the GB and SOTA methods. Albeit the estimated energy saving (Fig. 10) is slightly higher than the actual saving shown in Fig. 11, the difference is not significant. It confirms that the proposed method effectively reduces energy consumption in a real hardware platform.

7 Conclusion

We propose a scenario-based mapping methodology to run multiple DL applications on heterogeneous processors, aiming to reduce energy consumption while satisfying the real-time constraints. The proposed technique considers several technical challenges imposed by the use of NPU and its associated SDK. The proposed method consists of three steps. In the first step, we obtain the Pareto-optimal mappings for each application. In the second step, we seek a sub-optimal mapping combination of applications, considering the scenarios and real-time constraints. In the last step, we fine-tune the frequencies of processors if the deadline constraint is loosened. The proposed method is confirmed by using real-life applications with different scenarios. We could satisfy up to 40% higher deadline constraints and reduce the energy consumption by $22\% \sim 31\%$ compared to the state-of-the-art static mapping methods with real-life applications and different scenarios on a real platform. It is left to future work to apply the proposed method to new hardware platforms, SDKs, and applications that are continuously being developed.

References

[1] H. Zhou, S. Bateni, and C. Liu, "S^3DNN: Supervised streaming and scheduling for GPU-accelerated real-time DNN workloads," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp.*, 2018, pp. 190–201.
[2] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 392–405.
[3] D. Kang, J. Oh, J. Choi, Y. Yi, and S. Ha, "Scheduling of deep learning applications onto heterogeneous processors in an embedded device," *IEEE Access*, vol. 8, pp. 43 980–43 991, 2020.
[4] W. Kang, K. Lee, J. Lee, I. Shin, and H. S. Chwa, "Lalarand: Flexible layer-by-layer CPU/GPU scheduling for real-time DNN tasks," in *Proc. IEEE Real-Time Syst. Symp.*, 2021, pp. 329–341.
[5] NVIDIA TensorRT, 2022. Accessed: Jul. 15, 2022. [Online]. Available: https://developer.nvidia.com/blog/tensorrt-3-faster-tensorflow-inference/
[6] M. R. Garey, *Computers and Intractability; A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: Freeman, 1990.
[7] A. K. Singh, M. Shafique, A. Kumar, and J. Henkel, "Mapping on multi/many-core systems: Survey of current and emerging trends," in *Proc. IEEE/ACM/EDAC Des. Automat. Conf.*, 2013, pp. 1–10.

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.

[8] H. Oh, "A static scheduling heuristic for heterogeneous processors," in *Proc. Eur. Conf. Parallel Process.*, 1996, pp. 573–577.
[9] H. Topcuoglu, S. Hariri, and Min-You Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
[10] S.-H. Kang, D. Kang, H. Yang, and S. Ha, "Real-time co-scheduling of multiple dataflow graphs on multi-processor systems," in *Proc. IEEE/ACM/EDAC Des. Automat. Conf.*, 2016, pp. 1–6.
[11] E. A. Lee and D. G. Messerschmitt, "Synchronous data flow," *Proc. IEEE*, vol. 75, no. 9, pp. 1235–1245, Sep. 1987.
[12] M. Niknafs, "Runtime resource management with workload prediction," in *Proc. IEEE/ACM/EDAC Des. Automat. Conf.*, 2019, pp. 1–6.
[13] R. Khasanov and J. Castrillon, "Energy-efficient runtime resource management for adaptable multi-application mapping," in *Proc. Des. Automat. Test Eur. Conf. Exhib.*, 2020, pp. 909–914.
[14] B. Donyanavard, T. Mck, S. Sarma, and N. Dutt, "SPARTA: Runtime task allocation for energy efficient heterogeneous manycores," in *Proc. IEEE Int. Conf. Hardware/Softw. Codesign Syst. Synth.*, 2016, pp. 1–10.
[15] S. V. Gheorghita, "System-scenario-based design of dynamic embedded systems," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 14, pp. 1–45, 2009.
[16] L. Schor, "Scenario-based design flow for mapping streaming applications onto on-chip many-core systems," in *Proc. Int. Conf. Compilers, Archit. Synth. Embedded Syst.*, 2012, pp. 71–80.
[17] H. Jung, "Dynamic behavior specification and dynamic mapping for real-time embedded systems: Hopes approach," *ACM Trans. Embedded Comput. Syst.*, vol. 13, pp. 1–26, 2014.
[18] W. Quan and A. D. Pimentel, "A scenario-based run-time task mapping algorithm for MPSoCs," in *Proc. IEEE/ACM/EDAC Des. Automat. Conf.*, 2013, pp. 1–6.
[19] W. Quan, "Scenario-based run-time adaptive MPSoC systems," *J. Syst. Archit.*, vol. 62, pp. 12–23, 2016.
[20] B. Fang, "NestDNN: Resource-aware multi-tenant on-device deep learning for continuous mobile vision," in *Proc. Annu. Int. Conf. Mobile Comput. Netw.*, 2018, pp. 115–127.
[21] A. Mathur, "Deepeye: Resource efficient local execution of multiple deep vision models using wearable commodity hardware," in *Proc. Annu. Int. Conf. Mobile Syst. Appl. Serv.*, 2017, pp. 68–81.
[22] R. Pujol, "Generating and exploiting deep learning variants to increase heterogeneous resource utilization in the nvidia xavier," in *Proc. Euromicro Conf. Real-Time Syst.*, 2019.
[23] APOLLO: An open autonomous driving platform, 2018. Accessed: Mar 15, 2022. [Online]. Available: https://apollo.auto/
[24] E. Jeong, J. Kim, and S. Ha, "Tensorrt-based framework and optimization methodology for deep learning inference on jetson boards," *ACM Trans. Embedded Comput. Syst.*, vol. 14, pp. 15–18, 2022.
[25] T. Amert, N. Otterness, M. Yang, J. H. Anderson, and F. D. Smith, "GPU scheduling on the NVIDIA TX2: Hidden details revealed," in *Proc. IEEE Real-Time Syst. Symp.*, 2017, pp. 104–115.
[26] M. Yang, "Avoiding pitfalls when using NVIDIA GPUs for real-time tasks in autonomous systems," in *Proc. Euromicro Conf. Real-Time Syst.*, 2018, Art. no. 20.
[27] NVIDIA Jetson Developer Guide, 2022. Accessed: Jul. 15, 2022. [Online]. Available; https://docs.nvidia.com/jetson/archives/l4t-archived/l4t-3231/
[28] E. Tang, S. Minakova, and T. Stefanov, "Energy-efficient and high-throughput CNN inference on embedded CPUs-GPUs MPSoCs," in *Proc. Int. Conf. Embedded Comput. Syst.*, 2022, pp. 127–143.
[29] J. Palencia Gutierrez, J. J. Gutierrez Garcia, and M. Gonzalez Harbour, "On the schedulability analysis for distributed hard real-time systems," in *Proc. Euromicro Workshop Real Time Syst.*, 1997, pp. 136–143.
[30] J. Lehoczky, "Fixed priority scheduling of periodic task sets with arbitrary deadlines," in *Proc. IEEE Real-Time Syst. Symp.*, 1990, pp. 201–209.
[31] F. Eisenbrand and T. Rothvo, "Static-priority real-time scheduling: Response time computation is NP-hard," in *Proc. IEEE Real-Time Syst. Symp.*, 2008, pp. 397–406.
[32] M. Verucchi et al., "A systematic assessment of embedded neural networks for object detection," *Proc. IEEE Int. Conf. Emerg. Technol. Factory Automat.*, 2020, pp. 937–944.
[33] F.-A. Fortin et al., "DEAP: Evolutionary algorithms made easy," *J. Mach. Learn. Res.*, vol. 13, pp. 2171–2175, 2012.
[34] T. Helmuth, L. Spector, and J. Matheson, "Solving uncompromising problems with lexicase selection," *IEEE Trans. Evol. Comput.*, vol. 19, no. 5, pp. 630–643, Oct. 2015.
[35] J. Redmon et al., "Yolo9000: Better, faster, stronger," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7263–7271.
[36] J. Redmon and A. Farhad, "Yolov3: An incremental improvement," 2018, *arXiv:1804.02767*.



Jangryul Kim received the BS degree in computer science and engineering from Sogang University, Seoul, South Korea, in 2017. He is currently working toward the PhD degree with the department of computer science and engineering, Seoul National University, Seoul. His current research interests include system performance estimation and design space exploration of embedded systems.



Soonhoi Ha (Fellow, IEEE) received the BS and MS degrees in electronics engineering from Seoul National University, Seoul, South Korea, in 1985 and 1987, respectively, and the PhD degree in electrical engineering and computer science from the University of California at Berkeley, Berkeley, CA, USA, in 1992. He is currently a professor with Seoul National University. His current research interests include HW/SW codesign of embedded systems, embedded machine learning, and Internet of Things. He has actively participated in the premier international conferences in the EDA area, for instance serving CODES+ISSS in 2006, ASPDAC in 2008 as the program co-chair, and ESWEEK in 2018 as the general chair.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Zhejiang University. Downloaded on September 07,2026 at 01:35:16 UTC from IEEE Xplore. Restrictions apply.